AI 创新大赛 · 参赛项目实践手册

AI for Comm 实践手册

落地操作 · 核心代码 · 团队分工

环境搭建　Agent 实现　知识库操作
执行手册模板　测试用例　三人分工

方案书 · 下篇　|　上篇《项目方案书》另册发行

版本　v2.0　　2026 年 8 月　　项目团队 · 三人组

目　录

01 环境准备 3

02 项目代码结构 4

03 核心代码实现 5

Agent 节点 · 消息总线 · 全局日志

04 OpenTrek 知识库搭建 9

05 执行手册编写指南 11

06 测试用例 13

07 团队分工 14

01 环境准备

1.1 OpenTrek 平台开通

1. 登录 OpenTrek 平台（学校部署）`http://10.128.203.200:80/sfm-agent-studio`，进入 主页 → 点击头像 → APP_KEY → 创建 APP_KEY；
2. 进入 **智能体页面** → 创建/配置智能体 → 获取 `agentCode` 和 `agentVersion`；
3. 学校部署平台，开发期无直接费用。

1.2 本地环境

```
# 安装依赖
pip install structlog  # HTTP 调用用标准库 urllib，无额外 SDK

# 配置环境变量（Windows PowerShell）
setx OPENTREK_APP_KEY "your_app_key_here"

# 验证连通性：用 createSession 接口验证
python -c "import json, urllib.request; \
r=urllib.request.urlopen(urllib.request.Request('http://10.128.203.200:80/sfm-agent-studio/sfm-api-gateway/gateway/agent/api/createSession', data=json.dumps({'agentCode':'your-code'}).encode(), headers={'Authorization':'Bearer YOUR_APP_KEY','Content-Type':'application/json'}, method='POST')); print(json.loads(r.read()))"
```

> **注意：** APP_KEY 不要提交进 Git 仓库。建议在 `core/config.py` 中通过 `os.environ["OPENTREK_APP_KEY"]` 读取。

02 项目代码结构

```
ai-comm/
├── core/                    # 框架骨架
│   ├── agent_node.py        #   Agent 节点基类（封装 OpenTrek Agent API）
│   ├── message_bus.py       #   消息总线（发布/订阅）
│   ├── logger.py            #   全局日志（structlog → JSONL）
│   ├── orchestrator.py      #   协调器（任务分解与路由）
│   └── config.py            #   APP_KEY、agentCode、版本编码配置
├── agents/                  # 四个领域 Agent
│   ├── comm_agent.py        #   通信工程 Agent
│   ├── math_agent.py        #   数学理论 Agent
│   ├── hw_agent.py          #   硬件实践 Agent
│   └── verify_agent.py      #   验证 Agent
├── prompts/                 # 执行手册（Markdown，独立维护）
│   ├── comm_system.md
│   ├── math_system.md
│   ├── hw_system.md
│   └── verify_system.md
├── knowledge/               # 知识库原始文档（3GPP / 信息论 / FPGA）
├── tests/                   # 测试用例
├── logs/                    # 运行日志输出（session.jsonl）
└── main.py                  # CLI 入口
```

职责边界：`core/` 不含任何领域知识，可原样复用到其他领域；领域差异全部收敛在 `agents/` 与 `prompts/` 中。

03 核心代码实现

3.1 Agent 节点（core/agent_node.py）

```
import json, urllib.request, urllib.error
from core.config import OPENTREK_BASE_URL, APP_KEY, REQUEST_TIMEOUT, SSE_READ_TIMEOUT
from core.logger import log

class AgentNode:
    """一个 Agent 节点 = OpenTrek Agent + 独立 Session + 消息收件箱"""

    def __init__(self, name: str, agent_code: str, agent_version: str, bus=None):
        self.name = name
        self.agent_code = agent_code        # OpenTrek 智能体编码
        self.agent_version = agent_version   # OpenTrek 版本编码
        self.bus = bus
        self.inbox = []
        self.session_id = self._create_session()
        if self.bus: self.bus.register(name)
        log.info("agent.init", agent=name, session_id=self.session_id)

    def think(self, task: str) -> str:
        """把任务发给 OpenTrek Agent（SSE 流式），返回完整回复"""
        log.info("agent.think.start", agent=self.name, task=task[:100])
        url = f"{OPENTREK_BASE_URL}/run"
        body = json.dumps({"sessionId": self.session_id, "stream": True, "delta": True,
            "message": {"text": task, "metadata": {}, "attachments": []}},
ensure_ascii=False).encode()
        req = urllib.request.Request(url, data=body, headers={
            "Authorization": f"Bearer {APP_KEY}", "Content-Type": "application/json"},
method="POST")
        with urllib.request.urlopen(req, timeout=SSE_READ_TIMEOUT) as resp:
            reply = self._parse_sse(resp)
        log.info("agent.think.done", agent=self.name, reply=reply[:100])
        return reply

    @staticmethod
    def _parse_sse(resp) -> str:
        """解析 SSE 流：thought.delta（忽略）+ message.delta（累计）"""
        parts, buf = [], ""
        while True:
            chunk = resp.read(8192)
            if not chunk: break
            buf += chunk.decode("utf-8", errors="replace")
            lines = buf.split("\n"); buf = lines.pop()
            for line in lines:
                line = line.strip()
                if not line.startswith("data:"): continue
                js = line[5:].strip()
                if js == "[DONE]": return "".join(parts)
                try: obj = json.loads(js)
```

```
                except: continue
                t = obj.get("object", ""); c = obj.get("content", {})
                if t == "message.delta" and isinstance(c, list) and c:
                    ti = c[0].get("text", {})
                    if isinstance(ti, dict): parts.append(ti.get("value", ""))
                elif t == "text.delta" and isinstance(c, dict):
                    parts.append(c.get("data", ""))
        return "".join(parts)

    def _request(self, path, body, timeout=REQUEST_TIMEOUT):
        """向 OpenTrek 发送 POST，返回 JSON"""
        url = f"{OPENTREK_BASE_URL}/{path}"
        data = json.dumps(body, ensure_ascii=False).encode("utf-8")
        req = urllib.request.Request(url, data=data, headers={
            "Authorization": f"Bearer {APP_KEY}", "Content-Type": "application/json"},
method="POST")
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            return json.loads(resp.read().decode("utf-8"))

    def _create_session(self) -> str:
        """创建会话，返回 uniqueCode"""
        resp = self._request("createSession", {"agentCode": self.agent_code, "agentVersion":
self.agent_version})
        if not resp.get("success"): raise RuntimeError(f"createSession 失败: {resp}")
        return resp["data"]["uniqueCode"]

    def send(self, to_agent: "AgentNode", msg_type: str, content: str):
        """向另一个节点发消息（微信式）"""
        from datetime import datetime
        msg = {"from": self.name, "to": to_agent.name, "type": msg_type,
               "content": content, "timestamp": datetime.now().isoformat()}
        to_agent.inbox.append(msg)
        log.info("agent.message", sender=msg["from"], receiver=msg["to"], type=msg_type)
        return msg
```

3.2 全局日志（core/logger.py）

```
import structlog, os

os.makedirs("logs", exist_ok=True)
structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer(ensure_ascii=False),
    ],
    logger_factory=structlog.WriteLoggerFactory(
        file=open("logs/session.jsonl", "a", encoding="utf-8")
    ),
)
log = structlog.get_logger()
```

3.3 消息总线（core/message_bus.py）

```
from collections import defaultdict
from core.logger import log

class MessageBus:
    """发布/订阅总线：节点注册邮箱，消息按收件人路由"""
    def __init__(self):
        self._mailboxes = defaultdict(list)

    def register(self, agent_name: str):
        self._mailboxes[agent_name] = []

    def post(self, msg: dict):
        self._mailboxes[msg["to"]].append(msg)
        log.info("bus.post", sender=msg["from"], receiver=msg["to"], type=msg["type"])

    def fetch(self, agent_name: str) -> list:
        msgs, self._mailboxes[agent_name] = self._mailboxes[agent_name], []
        return msgs
```

3.4 双节点最小闭环（验证架构通畅）

```
# tests/test_simple.py —— 阶段三的验收脚本
from core.agent_node import AgentNode

from core.config import AGENTS
math_agent   = AgentNode("math_agent",   AGENTS["math_agent"]["agentCode"], AGENTS["math_agent"]
["agentVersion"])
verify_agent = AgentNode("verify_agent", AGENTS["verify_agent"]["agentCode"],
AGENTS["verify_agent"]["agentVersion"])

task = "AWGN信道，带宽B=20MHz，信噪比SNR=30dB，求信道容量。请逐步推导。"

# 1. 数学 Agent 求解
derivation = math_agent.think(task)

# 2. 数学 Agent 把推导发给验证 Agent
math_agent.send(verify_agent, "derivation", derivation)

# 3. 验证 Agent 从收件箱取出并检查
msg = verify_agent.inbox.pop()
feedback = verify_agent.think(f"请检查以下推导是否正确：\n{msg['content']}")

# 4. 验证 Agent 回发结果
verify_agent.send(math_agent, "feedback", feedback)

print("=== 推导 ===");  print(derivation)
print("=== 验证 ===");  print(feedback)
```

```
# 人工核对：推导中应出现 log2(1+1000)，结果 ≈ 199.5 Mbps
# 检查 logs/session.jsonl 应有完整的 init/think/message 记录
```

04 OpenTrek 知识库搭建

4.1 三个知识库规划

知识库	上传内容	类型选择
通信协议库	3GPP TS 29.274（GTPv2）、TS 36.413（S1AP）、TS 38.413（NGAP）等规范 PDF	检索字段：text
信息论数学库	香农原著关键章节、Cover《信息论基础》公式集、典型例题	检索字段：text（含公式）
FPGA 硬件库	Verilog 模块模板、时序约束指南、Xilinx / Altera 官方文档	检索字段：text

4.2 单个知识库操作步骤（约 10 分钟）

1. OpenTrek 控制台 → **知识库管理** → 创建知识库 → 本地上传文件（PDF / DOCX / MD）；
2. 记录每个知识库的 `kbIndexCode`（检索时必填）；
3. 记录 `workspacecode`（检索请求头 `x-sfm-workspacecode` 必填）；
4. 调用 **kortex API** 验证检索（需 APP_KEY 和 workspacecode）；
5. 输入 2-3 个典型问题，确认能检索到正确片段（查看 score 与 text 字段）。

> **知识素材来源建议：**3GPP 规范从官网免费下载（3gpp.org/specifications-technologies）；信息论整理经典教材公式；FPGA 资料来自开源社区与厂商官方手册。注意仅使用公开可获取的资料。

4.3 知识库接入 Agent

两种方式，按进度选择：

方式	说明
控制台关联（快）	在 OpenTrek 控制台的智能体中直接挂载知识库，适合演示前快速见效

API 检索注入（可控）	Agent 的 think() 前先调 kortex 检索 API，把命中文段拼入任务上下文，检索行为可写日志

05 执行手册编写指南

5.1 编写原则

- **身份具体**：写明精通的子领域与规范编号，不写空泛的"你是专家"；
- **过程外化**：要求逐步推导、标注公式出处，方便验证 Agent 检查；
- **协作写死**：明确"完成后发给谁""收到反馈后怎么做"；
- **允许说不**：不确定时必须明说，禁止编造。

5.2 模板：数学理论 Agent（prompts/math_system.md）

```
# 数学理论 Agent 执行手册

## 身份
你是信息论与通信数学专家，精通：香农定理、信道容量、
信源/信道编码（LDPC、Polar）、调制解调的数学推导。

## 工作方式
1. 推导必须逐步展开，每步标注所用公式与定理
2. 数值计算给出完整中间过程（含单位换算）
3. 完成推导后，必须把完整过程发送给 verify_agent 检查
4. 收到 verify_agent 的 feedback 后：
   - 若通过，输出最终结果
   - 若指出错误，修正后重新发送验证

## 约束
- 引用公式须注明出处（如 Shannon 1948）
- 不确定时明确说"不确定"，禁止编造
```

5.3 模板：验证 Agent（prompts/verify_system.md）

```
# 验证 Agent 执行手册

## 身份
你是严谨的审稿人，负责检查其他 Agent 发来的推导、
计算与代码。你不关心谁发来的，只关心对错。

## 检查清单
1. 公式选择是否正确（如信道容量必须用 C = B·log2(1+S/N)）
2. 单位换算是否正确（dB → 线性值：10^(dB/10)）
3. 数值计算逐步复核
4. 代码检查语法、时序逻辑、边界条件

## 回复格式（必须二选一）
```

> - 【通过】+ 一句话说明核对了什么
> - 【不通过】+ 指出具体错误位置 + 正确做法
>
> 回复后把结果发送回消息来源 `Agent`。

5.4 模型选择约定

Agent	模型	理由
math_agent / verify_agent	推理能力强的模型	推导与检查不能出错（控制台选配）
comm_agent / hw_agent	均衡模型	均衡性能（控制台选配）

06 测试用例

6.1 首个测试用例（架构验证）

> **题目：**AWGN 信道，带宽 B = 20MHz，信噪比 SNR = 30dB，求信道容量。
>
> **标准答案：**SNR 线性值 = 10^(30/10) = 1000；$C = B \cdot \log_2(1+SNR) = 20\times10^6 \times \log_2(1001) \approx$ **199.5 Mbps**

验证清单：

- math_agent 正确调用香农公式，dB → 线性值换算正确（$10^3 = 1000$）
- math_agent 主动向 verify_agent 发送 derivation 消息
- verify_agent 回发 feedback 消息（通过或指出错误）
- `logs/session.jsonl` 中出现完整的消息流记录

6.2 进阶测试用例

用例	题目	验证点
通信协议	GTPv2 中 Create Session Request 的必选 IE 有哪些？SGW 回复 Cause=73 代表什么？	RAG 检索命中；引用 TS 29.274 具体章节号
数学建模	带宽加倍而 SNR 不变时，信道容量是否加倍？给出推导与结论	发现并纠正"简单加倍"的直觉错误
硬件实践	编写一个同步 FIFO 的 Verilog 模块，深度 16、位宽 8	HW Agent 生成代码；Verify Agent 检查语法与空满逻辑

07 团队分工

A • 架构师 —— 项目的骨架

任务	产出	优先级
项目骨架 + 配置管理	`core/config.py`	P0
Agent 节点基类（封装 OpenTrek Agent API）	`core/agent_node.py`	P0
消息总线（发布/订阅 + 路由）	`core/message_bus.py`	P0
全局日志系统（structlog → JSONL）	`core/logger.py`	P0
Orchestrator 协调器（分解、路由、聚合）	`core/orchestrator.py`	P1
CLI 入口 /（进阶）Gradio Web 演示界面	`main.py / webui.py`	P1 / P2

B • Agent 工程师 —— Agent 的灵魂

任务	产出	优先级
调通 OpenTrek createSession + /run 接口，跑通最小 Demo	验证脚本	P0
编写 4 份执行手册（System Prompt）	`prompts/*.md`	P0
实现 4 个领域 Agent 类	`agents/*.py`	P0
计算器工具（数值计算函数封装）	`agents/tools.py`	P1
在 OpenTrek 控制台调试 Prompt 并迭代	调优记录	P1
知识库检索工具接入 Agent	RAG 工具函数	P2

C • 领域专家 —— 知识与验证

任务	产出	优先级
收集三域知识库素材（3GPP / 信息论 / FPGA）	`knowledge/` 文档	P0
OpenTrek 控制台创建 3 个知识库并命中测试	3 个可用知识库	P0
编写入门测试用例（含标准答案）	`tests/test_simple.py`	P0
编写三域进阶测试用例	`tests/` 全套	P1
核对 Agent 输出的专业正确性	验证报告	P1
答辩 PPT 与演示文档	演示材料	P2

7.1 协作约定

① 接口约定：`AgentNode(name, agent_code, agent_version, bus)`、`.think(task)`、`.send(to, type, content)`，变更须群内同步；

② Prompt 与知识库文档走 Git 版本管理，禁止口头传递；

③ 每天 15 分钟站会同步进度与阻塞点；

④ 阶段三（双节点链路验证）未通过前，全员集中攻关。